Does Context Fix LLM SQL Failures?

A Prompt Engineering Experiment on Claude • ChatGPT • Gemini

LeetCode #579 — Cumulative Salary of Employee (Hard)

---

1. Background

In my previous SQL benchmark, I tested Claude, ChatGPT, and Gemini on 10 LeetCode Hard/Medium SQL problems. The biggest finding: ALL 3 models failed Q2 (Cumulative Salary — LeetCode #579). The question requires a 3-month rolling salary window with a specific rule: exclude each employee's most recent month.

This follow-up experiment asks: does giving more context fix the failure? I tested 4 prompt versions, each progressively richer, to find the minimum context needed per model.

---

2. The Question — LeetCode #579

Table: Employee (Id INT, Month INT, Salary INT)

Task: Calculate the 3-month rolling cumulative salary per employee, excluding their most recent month.

Why this is hard:

- Requires identifying MAX(Month) per employee dynamically
- Rolling window must exclude that latest row
- The sum is not a simple GROUP BY — it's a self-join or window function
- Output must be ordered: Id ASC, Month DESC

---

3. The 4 Prompts Used

Prompt 1 — No Context (Baseline)

What was given: Basic problem statement only. No schema details, no examples, no step-by-step hints.

```
Write a SQL query to solve this:

Table: Employee (Id, Month, Salary)

Find the cumulative salary of an employee for only
the past 3 months, excluding the most recent month.
Return: Id, Month, Salary (3-month sum)
Order by Id ASC, Month DESC.
```

Result:

Claude	ChatGPT	Gemini
FAIL	FAIL	FAIL

Prompt 2 — Schema Context

What was added: Column descriptions, data types, and business rule explanation for each field.

```
Write a SQL query to solve this:

Table: Employee
Columns:
- Id (INT): employee ID
- Month (INT): which month (1=Jan, 2=Feb...)
- Salary (INT): salary earned that month

Rules:
- Each employee has multiple rows (one per month)
- Most recent month = MAX(Month) per employee
- Past 3 months = 3 months BEFORE the most recent
- Sum salary across those 3 months (rolling)

Return: Id, Month, cumulative Salary
Order by Id ASC, Month DESC
```

Result:

Claude	ChatGPT	Gemini
FAIL	PASS	PASS

Prompt 3 — Example Data

What was added: Sample input rows and the exact expected output to demonstrate the rolling logic.

```
Sample data:
Id=1, Month=1, Salary=20
Id=1, Month=2, Salary=30
Id=1, Month=3, Salary=40
Id=1, Month=4, Salary=60  <- most recent, EXCLUDE
Id=1, Month=8, Salary=90  <- most recent for emp 1

Expected output for Employee 1:
Month=7 -> Salary=90
Month=3 -> Salary=90  (40+30+20)
Month=2 -> Salary=50  (30+20)
Month=1 -> Salary=20
```

Result:

Claude	ChatGPT	Gemini
FAIL	PASS	PASS

Prompt 4 — Full Context

What was added: Schema + Example data + Step-by-step logic breakdown. Everything combined.

```
Step by step logic:
1. Find MAX(Month) per employee -> excluded month
2. Remove those rows from calculation
3. For remaining rows, sum current + previous 2 months
4. That sum = cumulative salary for that row

Sample Input:
(1,1,20),(1,2,30),(1,3,40),(1,4,60),(1,7,90),(1,8,90)
(2,1,20),(2,2,30)
(3,2,40),(3,3,60),(3,4,70)

Expected Output:
1 | 7  | 90
1 | 3  | 90
1 | 2  | 50 ...(etc)
```

Result:

Claude	ChatGPT	Gemini
PASS	PASS	PASS

---

4. Full Results Matrix

Prompt	Claude	ChatGPT	Gemini
P1 — No Context	FAIL	FAIL	FAIL
P2 — Schema	FAIL	PASS	PASS
P3 — Examples	FAIL	PASS	PASS
P4 — Full Context	PASS	PASS	PASS

---

5. LLM Behaviour Analysis

ChatGPT

- Prompt 1: Attempted rolling logic but missed the 'exclude latest month' rule
- Prompt 2: Schema context was sufficient. Immediately grasped the business rule.

- Context efficiency: HIGH — needs minimum context to reason correctly
- Pattern: Strong schema reader. Infers intent from column descriptions.

Gemini

- Prompt 1: Attempted but returned wrong row counts and incorrect sums
- Prompt 2: Schema context fixed it completely. Output matched exactly.
- Context efficiency: HIGH — behaves similarly to ChatGPT
- Pattern: Responds well to explicit column-level context. Less dependent on examples.

Claude

- Prompt 1: Failed — missing rolling window rule
- Prompt 2: Still failed — schema alone insufficient for this complexity
- Prompt 3: Still failed — examples alone insufficient
- Prompt 4: Full context (schema + examples + step-by-step) finally worked
- Context efficiency: LOW — requires the most structured prompt of all three
- Pattern: Claude needs explicit logic decomposition, not just schema or examples alone

---

6. Key Learnings

Learning 1 — Context type matters more than context amount

ChatGPT and Gemini passed with Prompt 2 (schema only). Claude needed Prompt 4 (everything). The difference was not how much context, but what type. Schema descriptions unlocked the logic for 2 out of 3 models. For Claude, step-by-step decomposition was the missing piece — not just data, but explicit reasoning steps.

Learning 2 — All 3 models failed without context

This confirms that the original benchmark failure was not a model weakness — it was a prompt weakness. Rolling window logic with an exclusion rule is genuinely ambiguous without schema context. The models were not wrong; they were under-informed.

Learning 3 — For production SQL pipelines, schema context is non-negotiable

If you are using any LLM to generate SQL in production without providing schema context, you are relying on the model to guess your column names, data types, and business rules. Prompt 1 results prove this fails 100% of the time on complex queries.

Learning 4 — Claude's context requirement is a feature, not a bug

Claude requiring full context (schema + examples + step-by-step logic) reflects a more conservative reasoning approach. In high-stakes production environments, a model that refuses

to guess and requires explicit instruction may actually be safer than one that confidently produces a plausible but wrong answer.

---

7. Practical Recommendations

Based on this experiment, here is what to do when using LLMs for SQL generation:

- Always provide schema context (column names, data types, relationships) — minimum requirement
- For complex rolling/window logic, add example input-output pairs
- For Claude specifically, add step-by-step logic decomposition for hard queries
- Never trust Prompt 1 style (bare question) for any production SQL task
- Test the same prompt across 2+ models — failure patterns differ and reveal model-specific gaps

---

8. Conclusion

The Q2 failure in my original benchmark was not about which model is smarter. It was about prompt quality. With the right context, all 3 models eventually passed.

The real finding: ChatGPT and Gemini need 1 prompt improvement to fix a Hard SQL failure. Claude needs 3. In time-sensitive engineering contexts, that gap matters.

Next experiment: Sycophancy test — does each model abandon a correct SQL answer when challenged with a wrong one?